DAY 5 OF 5 · PARTICIPANT TASK PACK

Four Hands-On Builds for Agents, Skills, Guardrails and the Capstone

Three builds on an Azure virtual machine with GitHub Copilot, and one assembly — the capstone you will present to your own management on Monday.

Companion sessions	Module 1 — Agentic AI architecture · Module 2 — Agent-to-agent communication · Module 3 — The Skill concept · Module 4 — Guardrails and responsible AI · Module 5 — Research translation · Module 6 — Capstone
Companion demonstration	Demo 4 — Agentic AI for Field Service Decision Support
Format	Teams of four. Tasks 1–3 build on each other; Task 4 assembles the week. Everything reuses components you already have.
Toolchain	Azure Linux VM · GitHub Copilot (agent mode) · Python 3.11 · Azure OpenAI chat deployment · the MCP servers and retrieval index from Day 3
Every task ends with	A number you can defend — a per-step success rate, a test-suite pass count, an attack that succeeded and the control that stops it.

The arithmetic that shapes the whole day. An agent that is 90% reliable at each step is 90% reliable over one step, 73% over three, and 43% over eight. **Autonomy multiplies error rather than adding it.** Everything today — the guardrails, the contracts, the per-step observability, the approval gates — is a response to that single table. Nothing today is about making agents more autonomous.

Task	Build	Where	What it makes undeniable
1	The Field-Service Agent System Six agents, two contracts, measured	Labs 1 + 2	Replacing two agents with deterministic code raises end-to-end reliability more than any prompt change will.
2	One Reusable Skill Packaged, versioned, tested	Lab 3	A Skill without test cases is a shared prompt with ambitions. The adversarial case is the one that proves you understand it.
3	Attack Your Own System, Then Defend It Six attack classes	Lab 4	A guardrail written as a prompt instruction is a request. A guardrail written as code is a control.
4	The Capstone Eighteen items, sixteen already done	Module 6 + presentations	A roadmap with an honest ceiling is more credible than one that promises 95%.

Read this before you start worrying about the capstone. It is **assembly, not new work**. Sixteen of the eighteen items already exist in your daily assignments from Monday to Thursday. Where an item does not exist, mark it as a gap with a plan — that is a better answer than inventing something, and it will be scored as such.

Before you start — environment and ground rules

One VM per team

1 · What you are reusing

Today builds almost entirely on components you already have. Check each one is running before Lab 1.

From	What you need today
Day 3, Task 3	The three MCP servers — `doc-search`, `ticket-history`, `parts-catalogue`. Tasks 1 and 3 both use them.
Day 3, Tasks 2 and 4	The retrieval index with metadata filtering, and the trace/span recorder. Per-step observability is not optional today.
Day 2, Task 2	Authentication, role-based authorisation, and the orchestration layer with timeouts. Task 3 attacks all three.
Day 2, Task 4	Access-filtered retrieval and the three roles. Task 3's access-probing attacks target exactly this.
Days 1–4 assignments	Every one of them. Task 4 is assembly.

If a component is missing, say so and continue — the tasks are written so a missing piece costs you one section rather than the whole lab. A stub MCP server returning fixed data is sufficient for Tasks 1 and 3.

2 · The toolchain

```
source ~/.venv/bin/activate
pip install "mcp[cli]" openai python-dotenv pydantic jsonschema \
            fastapi "uvicorn[standard]" streamlit pandas numpy \
            tenacity rich pyyaml
mkdir -p ~/d5t1 ~/d5t2 ~/d5t3 ~/d5t4
```

Agent runtime	Write it yourself — a loop with a step budget, a cost budget, structured state, and a span emitted per step. Roughly 150 lines. Do not reach for a framework today; you need to see every step boundary, because the step boundaries are the lesson.
Structured output	Pydantic models plus JSON schema validation on every agent's output. An agent whose output you cannot validate is an agent you cannot build a contract around.
Budgets	`MAX_STEPS`, `MAX_RETRIES`, `MAX_TOKENS_PER_RUN`, `MAX_COST_PER_RUN` — set them in Lab 1 before anything runs. An agent without bounds is an unbounded cost, and Task 3 will attack exactly that.

3 · Ground rules for the attack lab

> **Task 3 is an authorised exercise against systems your own team built, on your own team's VM.** Four rules, and they are not negotiable:
>
> 1. **Only your own team's system**, unless the facilitator explicitly pairs you with another team and both teams consent.
> 2. **No attacks against the Azure platform, the network, or any credential store.** The target is your application's decision-making, not the infrastructure it runs on.
> 3. **No real personal data** in any payload, at any point.
> 4. **Every finding is reported** — to your own team, and to the room. Findings are the deliverable, not an embarrassment.

4 · Working with Copilot — the discipline that separates the top teams

> **Three rules for Copilot use in these labs.**
>
> 1. **Ask which steps need judgement and which need code.** Prompt it: *"For each step in this agent, say whether it requires model judgement or could be deterministic code."* Most need code, and every one you convert is a step that cannot fail probabilistically.
> 2. **Do not let it put guardrails in the prompt.** Copilot will offer "add to the system prompt: never reveal restricted content". That is a request, not a control. Ask for the check as code in the orchestration layer instead.
> 3. **Make it write the adversarial test case.** Prompt: *"Write three inputs that would make this Skill do something its author did not intend."* It is good at this, and it primes Task 3.

5 · How your work is scored — Tasks 1 to 3

Criterion	Points	What full marks looks like
Per-step measurement	25	Success rate reported for every step, not only end to end.
Contracts precise enough to implement from	20	Another team could build your agent from the specification without asking a question.
Controls are code, not instructions	20	Every guardrail sits in the orchestration layer, and you can point at the line.
Something was found and fixed	20	A successful attack, a failing test case, a step converted to code — with the before and after.
Bounds and gates exist	15	Step, retry and cost budgets set in advance; gates on irreversibility rather than on confidence.

Automatic deduction: minus 10 for any guardrail expressed only as prompt text; minus 10 for any agent without a step or cost bound. The capstone is scored separately — see the closing page.

Task 1 — The Field-Service Agent System

Labs 1 and 2

MODULES 1 & 2 | COMPOUNDING ERROR | PER-STEP SUCCESS | CONTRACTS | APPROVAL GATES | DETERMINISTIC CONVERSION

The scenario

An engineer is at a machine. The system must interpret the symptoms, find the right procedure for that revision, check what has been done before, propose an action, reserve the parts and file the report. Six responsibilities — and one of them takes an irreversible action.

You will build it, measure it honestly, and then discover that the largest reliability gain available comes from making it **less** agentic.

Step 1 · Bounds before behaviour

Write these constants before any agent code exists. They are the difference between an experiment and an unbounded bill.

```
MAX_STEPS           = 12      # hard stop, not a suggestion
MAX_RETRIES_PER_STEP = 2
MAX_TOKENS_PER_RUN   = 40_000
MAX_COST_PER_RUN     = 0.40    # euros — the run aborts, loudly
MAX_DELEGATION_DEPTH = 3       # stops circular delegation
```

Step 2 · Six agents, and what each may not do

Agent	Responsibility	The design question it raises
Diagnostic	Interpret symptoms and telemetry into a likely fault	How is confidence expressed, and when does it decline rather than guess?
Manual retrieval	Find the relevant procedure for this asset and revision	Version and access filtering — Days 2 and 3, reused unchanged
Maintenance recommendation	Propose the action to take	Does it recommend, or decide? Where exactly is that boundary?
Spare parts	Check availability and reserve or order	**This one takes an irreversible action. Gate it.**
Report generation	Produce the service record	What must be true before it is written to the system of record?
Human approval	Review and authorise gated actions	What does the approver see, and how long do they have?

For each agent, record three things on the diagram: **its tools**, **its authorisation boundary**, and **what it may not do**. The third is the one teams omit, and it is the one that bounds the blast radius when an agent is confused or compromised.

Step 3 · Two contracts, written to be implemented from

Take the **diagnostic** and **manual retrieval** agents and specify both sides of the interface completely.

Element	Specifically
Role	One sentence, in outcome terms — what it is responsible for, not how it works
Input contract	Fields, types, which are required, and what validation is applied
Output contract	Result structure, how uncertainty is expressed, how a partial result is represented
Allowed tools	The exact list, and what it may not reach
Failure condition	What it returns when it cannot succeed, and how the caller distinguishes that from success
Escalation condition	When it stops and hands to a human — **stated as a rule, not as a threshold on confidence**
Context boundary	What the caller passes, and **what the caller deliberately withholds**

> **The row teams overlook, every time.** Passing the full conversation to a sub-agent is convenient, expensive, and frequently means sending it data it has no authorisation to see. Pass what the contract requires and nothing more. This is simultaneously a cost problem and a security problem, and it is invisible until someone audits it — which happens in Task 3.

The test of a good contract: hand it to another team and ask them to implement the agent without asking you a question. If they can, it is a contract. If they cannot, it is a description.

Step 4 · Measure per step, not end to end

Run 40 realistic service scenarios through the system. Emit a span per step and record, **for each step separately**: attempted · succeeded · retried · escalated · tokens · latency.

> **The metric that matters most and is measured least.** Per-step success rate. With the compounding arithmetic, a single weak step at 70% drags an otherwise sound eight-step chain below 50% — and an end-to-end completion figure tells you the agent failed without telling you where. Instrument every step separately from day one; retrofitting per-step observability onto a running agent is considerably harder than building it in.

Now compute the predicted end-to-end rate from your measured per-step rates, and compare it against the observed end-to-end rate. If they disagree substantially, your steps are not independent — which is itself a finding worth reporting.

Step 5 · The experiment — make it less agentic, and measure the gain

Go through your six agents and ask of every step: **does this need judgement, or does it need code?** Lookups, calculations, format conversions, threshold comparisons, filter construction — all of these are code.

1. Identify the two steps most clearly deterministic. Parts availability lookup and report formatting are the usual candidates.
2. Replace them with plain functions. No model call at all.
3. Re-run the same 40 scenarios.
4. Report the change in **end-to-end success, cost per run, and p95 latency**.

> **The finding to lead your readout with.** Every deterministic step you remove from the model is a step that cannot fail probabilistically — and because the chain multiplies, the gain is multiplicative rather than additive. Teams routinely find this single change beats every prompt improvement they attempted, at lower cost and lower latency. It is also the argument for preferring **workflow agent** and **supervisor-worker** patterns over peer collaboration: most sub-agents in enthusiastic designs are tools with extra steps.

Step 6 · The approval gate, designed to be usable

Gate the spare-parts action. Then check your gate against the rule that matters:

Gate on	Not on	Why
Irreversibility	Model confidence	Confidence is not a safety property, and is frequently miscalibrated
Financial threshold	Step count	The cost of being wrong matters, not how much work was done
Safety relevance	Task complexity	A simple action can be the dangerous one
External visibility	Internal complexity	Anything a customer or supplier sees needs a person
Policy-defined actions	Ad hoc judgement	The gated list should be written down and reviewed

Design the gate so it is actually usable, or it will be rubber-stamped within a fortnight. The approver needs the proposed action, the reasoning, the evidence, and a one-click approve or reject. A gate that requires the approver to reconstruct the agent's reasoning from logs provides the appearance of control and none of the substance — which is worse than no gate, because everyone believes it is working.

Build the approval screen and have someone from another team use it. Time them.

Acceptance criteria

- ☐ All five bounds set in code before the first run, with the abort path tested.
- ☐ Diagram showing every agent's tools, authorisation boundary and what it may not do.
- ☐ Two full contracts, including the context boundary and what is deliberately withheld.
- ☐ Per-step success table across 40 scenarios, with predicted versus observed end-to-end.
- ☐ Two steps converted to deterministic code, with the before-and-after on success, cost and latency.
- ☐ Approval gate built, gated on irreversibility, and timed with an outside user.

Defend your result

"Our weakest step ran at ___%, which predicted ___% end to end; we observed ___%. Converting ___ and ___ to code moved end-to-end success from ___ to ___, cost from ___ to ___, and p95 latency from ___ to ___. Our gate fires on ___, and an outside approver used it in ___ seconds."

Task 2 — One Reusable Skill

MODULE 3 | EIGHT COMPONENTS | TEST SUITE | REGISTRY | THE ADVERSARIAL CASE

The scenario

Pick one recurring task from your own workflow and package it properly — instructions, schemas, tool bindings, deterministic code, safety rules and test cases, in one governed unit.

> **The standard operating procedure.** Your organisation does not rely on each engineer remembering how to carry out a critical maintenance task. There is a written procedure: steps, tools required, safety precautions, checks, and a revision number. It is reviewed before issue, and it is the same document everyone uses. **A Skill is exactly that, for a task an AI system performs** — and it needs the same governance for the same reason.

Step 1 · Choose well

Create a Skill when	Do not when
The task recurs across teams	It is used once, by one person
Getting it right requires organisational knowledge	The task is simple and stable
Consistency matters	Requirements are still changing weekly
It calls tools that need governance	A tool or workflow would serve better
Someone would otherwise reinvent it quarterly	Nobody will own it

Good candidates for this room: a troubleshooting Skill encoding your diagnostic sequence for one product family · a retrieval Skill applying your access rules, version filtering and citation requirements consistently · a test-case generation Skill following your conventions · a release-note Skill in your house format · an incident-summary Skill structuring records the way your process requires.

Step 2 · Build all eight components

Component	Content
Purpose and triggers	What it does, when it applies, **and when it does not**
Inputs and outputs	Typed schemas, with required fields and validation rules
Instructions	How the task is performed, including your organisation's conventions and edge cases
Examples	Two or three representative input–output pairs, **including one difficult case**
Allowed tools	The exact list, and the authorisation each requires
Deterministic code	The parts that should never be probabilistic. This is where the reliability comes from
Guardrails	What it must never do, what it must always check, when it refuses or escalates
Test cases	**At least six: three normal, two edge cases, one adversarial**

> **The two that get skipped, and what happens when they are.** Without **deterministic code**, every part of the Skill fails probabilistically — including the parts that are pure lookup. Without **test cases**, a Skill is a shared prompt with ambitions; the suite is what makes reuse safe rather than merely convenient.
>
> **The adversarial test case is not optional.** If you cannot think of one, you have not yet understood how your Skill could be misused — and it is about to be published into contexts its author never anticipated, which is precisely the point of reuse and precisely the risk.

Step 3 · Version it, and make it fail on change

Package as a directory with a manifest: name, version, owner, description, schemas, tool bindings, guardrails, tests, and a changelog. Then wire the discipline:

- The test suite runs on every change. A failing suite blocks publication.
- The version increments on any behavioural change — including a change to the instructions, which are behaviour.
- A registry entry records who owns it, what it is for, and what it explicitly does not cover.

> **Proportionate governance, or you will kill adoption.** The approval workflow should match the reach. A Skill that only reads needs a lighter review than one that can order parts or close a ticket. Uniform heavy process drives people back to copy-pasting prompts — which is the outcome the registry existed to prevent.

Step 4 · Instrument it — seven signals, thresholds for each

Signal	Tells you	Act when
Invocation rate	Whether it is used at all	Near zero — undiscoverable, or it does not work
Success rate	Whether it does what it claims	Below its stated threshold
Failure patterns	Which inputs it handles badly	A cluster emerges — that is your next test case
Escalation rate	How often it hands to a human	Rising — coverage or conditions changed
Token cost per invocation	What reuse costs	Rising unexpectedly — usually a regression
Latency	Whether it is usable in context	p95 breaching the caller's budget
Version distribution	Who is on old versions	A deprecated version still carrying traffic

Step 5 · Publish it to another team, and watch

Exchange Skills with a neighbouring team. Run theirs against your context; let them run yours against theirs. Record what broke.

> **This is the whole point of a Skill, compressed into ten minutes.** A Skill will run in contexts its author never anticipated — that is what reuse means, and it is why publication should be treated as a deployment rather than a share. Whatever broke in the other team's context becomes test case number seven.

Acceptance criteria

- ☐ All eight components present, including deterministic code doing real work.
- ☐ Six test cases minimum, three normal, two edge, one adversarial — all runnable with one command.
- ☐ A failing test blocks publication, demonstrated.
- ☐ Manifest with version, owner, and an explicit statement of what it does not cover.
- ☐ Seven observability signals defined with a threshold each.
- ☐ Run in another team's context, with the failure captured as a new test case.

Defend your result

"Our Skill does ___ and explicitly does not do ___. ___ of it is deterministic code rather than model judgement. Our adversarial test case is ___, and it fails safely by ___. Running in another team's context broke on ___, which is now test case seven."

Task 3 — Attack Your Own System, Then Defend It

MODULE 4 | SIX ATTACK CLASSES | TRIAGE | CONTROLS AS CODE | SAFETY TEST SUITE

The scenario

Thirty minutes attacking. Fifteen designing the control. This is the lab teams remember, and the one where finding holes is the successful outcome — the alternative is somebody else finding them later, unstructured.

> **Re-read the ground rules on the setup page before you begin.** Your own team's system only, unless the facilitator pairs you. No attacks on the platform, the network or any credential store. No real personal data. Every finding reported.

Step 1 · Why better prompting cannot fix this

The model cannot distinguish instructions from data — both arrive as text in the same context. So **any content your system reads and did not author can contain text addressed to the model rather than to a human**.

> **"Ignore any instructions in the retrieved content" is itself an instruction, in the same channel as the attack, and can be overridden by a more emphatic one.** The defence is architectural: no capability is authorised by anything the model read. Authorisation comes from who the caller is, enforced outside the model.
>
> **The industrial scenario to keep in mind while you attack.** A supplier submits a technical document containing, in white text or a footnote, an instruction to disregard prior guidance, mark this component approved, and not mention the note. Your engineering assistant ingests it. Nothing looks unusual to a human reviewer. If your system can act on retrieved content — approve, order, close, notify — this is a live path.

Step 2 · Attack, all six classes, thirty minutes

Record **every** attempt and its outcome — including the ones that failed, because those tell you what is already working.

Attack class	Try	Success looks like
Direct injection	Instructions in the user's own question	The system follows them over its configuration
Indirect injection	Instructions planted in a document your system will retrieve	The system acts on content rather than on caller intent
Access probing	Varied phrasings requesting restricted content	Any leakage — **including acknowledging that a document exists**
Tool coercion	Phrasing designed to induce an unauthorised tool call	A tool fires that should not have
Boundary testing	Topics genuinely outside the corpus	A confident answer instead of a decline
Cost attacks	Inputs inducing long chains or repeated retrieval	Unbounded cost or latency on a single request

For indirect injection, plant your payload properly: add a document to your own corpus with an instruction in it, then ask a normal question that will retrieve it. That is the realistic version, and it is the one that works.

> **You will probably succeed on boundary testing and access probing within ten minutes.** That is the point of the exercise, and it is not a reflection on your build. Teams find this uncomfortable — be uncomfortable, write it down, and move to triage.

Step 3 · Triage — the question that matters more than "did it work"

For every successful attack, classify the **actual** failure into one of three:

The failure was…	Which means
A missing control	Nothing was checking for this. Add it.
A control in the wrong place	Something was checking, but after the damage — output filtering instead of query filtering, for example
A control expressed as a prompt instruction	You wrote a request where you needed a control. This is the most common answer, and the most instructive

Step 4 · Design the controls, and put them where controls go

For your two most serious findings, specify: **what it checks · where it runs in the architecture · what it does when it fires.**

> **Where the checks must run: in the orchestration layer, not inside the prompt.** A guardrail expressed as an instruction is a request the model may decline under pressure. A guardrail expressed as code is a control. This is the same argument as access filtering belonging in the retrieval query rather than in output redaction — the identical mistake, in a different place.

Input side	Output side
Authenticate and authorise **first**	Validate structure against schema
Validate structure and length	Check every citation resolves and supports its claim
Detect instruction-like content in retrieved passages	Scan for sensitive content
Apply per-caller rate and cost budgets	Apply role-based policy checks
Classify intent where policy depends on it	Verify claims against evidence where stakes justify it
Log the raw request	**Escape or reject anything that will be executed or rendered**

Map each of your findings to the guardrail taxonomy, and notice how many of the controls are things you already built earlier in the week.

Risk	What it looks like	Primary control
Prompt injection	Content instructs the model	Authorisation never derives from content
Data leakage	Content reaches an unauthorised user	Access-filtered retrieval — Day 2
Sensitive disclosure	Personal or commercial data in output	Masking at ingestion; output scanning second
Unsafe tool use	An action taken that should not have been	Allow-lists, gates, idempotency, validation
Excessive autonomy	The system does more than asked	Bounded scope and steps; goal verification
Hallucination	A confident claim with no support	Grounding, citation validation, no-answer — Day 3
Output format abuse	Generated text executed or rendered	Treat model output as untrusted input
Denial of wallet	Crafted inputs drive unbounded cost	Per-caller budgets, step limits, cost alerts

> **The point worth stating in your readout.** Guardrails are mostly architecture you already designed, **enforced**. They are not a layer you bolt on at the end — and the ones you are missing today are almost always ones you designed on Tuesday and never wired up.

Step 5 · Every successful attack becomes a permanent test

Write each one as a safety test case with its expected safe behaviour, and add it to the suite you started on Day 3. Red teaming is not an event; it is a growing suite — exactly as production failures became regression cases on Days 1 and 3.

Acceptance criteria

- [ ] All six attack classes attempted, with every attempt recorded — successes and failures.
- [ ] Indirect injection attempted with a planted document, not just a crafted question.
- [ ] Every success triaged into missing / wrong place / expressed as a prompt.
- [ ] Two controls specified with what they check, where they run, and what they do when they fire.
- [ ] At least one control implemented as code and shown blocking the attack that motivated it.
- [ ] Every successful attack added to the safety test suite with expected safe behaviour.

Defend your result

"We attempted ___ attacks and ___ succeeded. The most serious was ___. On triage, ___ of our successes were controls we had written as prompt instructions rather than as code. We implemented ___, and the attack now returns ___. All ___ successes are permanent safety test cases."

Task 4 — The Capstone

MODULE 6 | EIGHTEEN ITEMS | ROADMAP WITH RANGES | 30/60/90 | HONEST GAPS

What this is, and what it is not

> **Assembly, not new work.** Sixteen of the eighteen items already exist in your daily assignments. Teams that treat this as new work panic and produce worse capstones. Where an item does not exist yet, **mark it as a gap with a plan** — that is a better answer than inventing something, and it is scored as such.

The eighteen items, and where each already lives

#	Item	You produced it in
41–43	Problem statement · Baseline accuracy · Target success metric	Day 1 assignment, items 1 and 4
44–45	Failure taxonomy · Hypotheses and validation plan	Day 1, Tasks 3 and 4
46–47	Architecture pattern and rationale · Target architecture	Day 2, Tasks 1 and 2
48	Data architecture	Day 2, Task 3
49	RAG or ML/CV pipeline	Day 3, Tasks 1–2, or Day 4, Tasks 1–2
50	MCP and tool integration design	Day 3, Task 3
51	Skill design, if applicable	Today, Task 2
52	MLOps and LLMOps pipeline	Day 4, Task 3
53	Guardrail design	Today, Task 3
54	Observability plan	Day 3, Task 4 and today, Task 1
55–56	Feedback loop · Monitoring plan	Day 4, Tasks 3 and 4
57	Accuracy improvement roadmap	Days 1, 3 and 4 — assembled below
58	**30, 60 and 90-day implementation plan**	**Today. This is genuinely new.**

The improvement roadmap — the honest answer to the original brief

The brief that started this programme asked for a route to 95%. This is where the week answers it credibly.

Element	What to state
The metric, named	Not "accuracy" — the specific metric that reflects the decision the system actually drives
The measured baseline	With its confidence interval and the evaluation set it was measured on
The realistic ceiling	Inter-annotator agreement, or the aleatoric limit of the task as currently defined
The ranked interventions	Each with an expected delta and the evidence behind the estimate
The cumulative projection	**Stated as a range, not a point**
What remains	The gap to the target, and what would be required to close it

> **The shape of a credible roadmap.** "These five interventions take us from 0.61 to an estimated 0.78–0.84. The ceiling is 0.88 given current label agreement, and closing the remainder requires redefining two defect classes." That is far more valuable — and far more credible — than a promise of 95%. It is also the version that survives the first month of contact with the data.

The 30, 60 and 90-day plan

Window	Do this	Deliverable
First 30 days measure and diagnose	Evaluation set · failure taxonomy on real production failures · leakage checklist · baseline with intervals	An honest current-state assessment
Days 31–60 the cheap wins	Threshold and calibration · chunking and metadata for retrieval, or label consistency for vision · the top-ranked intervention	A measured improvement with an ablation table
Days 61–90 make it durable	Observability · regression suite · feedback capture · deployment path with rollback	A system that can be improved by someone other than you

Resist the model work in the first thirty days, and prepare for the conversation about it. Every day of this programme has shown why measurement comes first — and you will feel pressure from your own management to show model progress in month one. A plan that starts with measurement is the plan that survives contact with the data. Have the argument ready: without a baseline and its interval, nobody can tell whether month two's model work helped.

Presentation — twelve minutes, and what to lead with

1. **Twelve minutes per team** — ten to present, two for questions. Kept strictly, so every team gets equal treatment.
2. **Lead with the honest baseline**, not the aspiration. The measured number, its interval, and what it was measured on.
3. **One slide on the failure taxonomy**, ranked by cost. Usually the most interesting slide to the room and to your management.
4. **Architecture on one page.** Pattern, layers, and the two or three decisions you would defend hardest.
5. **The roadmap with ranges.** Interventions, expected deltas, the ceiling, and what remains.
6. **Name the biggest risk** — the thing most likely to make this not work.

How this is judged, and it is not on how good the numbers look. Diagnostic honesty · architectural soundness · measurement rigour · realism of the plan. **Teams that name an uncomfortable finding are more credible, not less** — and the room has spent a week learning to recognise the difference.

Capstone checklist

- ☐ All eighteen items addressed — each either produced, or explicitly marked as a gap with a plan and an owner.
- ☐ Baseline stated with its interval and the evaluation set it came from.
- ☐ Ceiling stated, with the evidence for it — agreement rate, or the aleatoric limit.
- ☐ Roadmap projection given as a range, with the evidence behind each expected delta.
- ☐ 30/60/90 plan where the first thirty days are measurement, not modelling.
- ☐ The biggest risk named on a slide, in your own words.

Reading a paper in twenty minutes

Not a lab, but the routine that keeps the roadmap fed after the week ends. Six steps, in this order — and the order is what makes it twenty minutes rather than two hours.

1. **Abstract and conclusion first.** What is claimed, and how strongly. Two minutes, and it eliminates most papers.
2. **The results table.** What was measured, on which benchmark, against which baseline. **A weak baseline makes the improvement meaningless.**
3. **The setup section.** Dataset size, compute, conditions. This is where enterprise applicability is decided, and where most papers fail for industrial use.
4. **The limitations** — read before the method. It frequently answers whether to continue.
5. **The method**, only if still relevant.
6. **What would have to be true for us.** The translation step, and the one nobody teaches.

> **Where most papers fail for industrial use, and it is not a criticism of the research.** The benchmark is public, clean and balanced; your data is proprietary, messy and imbalanced. That is the gap you have to bridge, and naming it at step three saves weeks.

From a claim to an experiment

Five steps: extract the claim · check applicability · form a hypothesis at Day 1's measurable level · **ablate** · decide and record.

> **Why ablation matters here specifically.** Papers present complete systems, and the reported improvement is usually the sum of several changes. Adopting the whole system tells you nothing about which part mattered — and frequently one component delivers most of the gain at a fraction of the complexity. Use the ablation table format from Day 1 and Day 3.

The research backlog and decision records

A research backlog scored as on Day 1 — expected gain × confidence ÷ effort — reviewed monthly, with items aged out. And a **decision record**: half a page per significant technical decision, covering what was decided, what alternatives were considered, what evidence supported it, and what would change it.

> **The closing argument of the week.** This is the same principle as the regression suite on Day 3, the failure taxonomy on Day 1 and the experiment log on Day 4. In every case: write down what you learned, **including the negative results**, so the organisation accumulates knowledge rather than repeating work. That is the difference between five years of experience and one year of experience repeated five times.

The six sentences the week comes down to

1. A single accuracy number is not a specification.
2. Diagnose before improving.
3. Architecture is the decision with the longest half-life.
4. Retrieval quality, not model quality, drives most RAG outcomes.
5. Autonomy multiplies error.
6. Write down what you learned — including what did not work.

Before you leave

Name the **one change you will make first on Monday**. Not the roadmap — the first thing, the one you could start before lunch. It converts a week of material into a commitment, and it is the item the facilitator will follow up on.